In the name of God

UNIVERSITY OF TEHRAN

Artificial Intelligence

Parham Javan

ID: 810800008

CA 1

INTRO

Add necessary libraries

```
from collections import deque
import copy
import random
import heapq
from dataclasses import dataclass, field
from typing import List, Tuple, Callable, Any, Set
import numpy as np
from time import time
from tabulate import tabulate
import threading
import collections
from math import ceil
from enum import Enum
from functools import partial
from copy import deepcopy
```

NECASSARY FUNCTIONS TO MANIPULATE THE PUZZLE

```
class LightsOutPuzzle:
    def __init__(self, board: List[List[int]]):
        self.board = np.array(board, dtype=np.uint8)
        self.size = len(board)

    def toggle(self, x, y):
        for dx, dy in [(0, 0), (1, 0), (-1, 0), (0, 1), (0, -1)]:
            nx, ny = x + dx, y + dy
            if 0 <= nx < self.size and 0 <= ny < self.size:
                self.board[nx, ny] ^= 1

    def is_solved(self):
        return np.all(self.board == 0)

    def get_moves(self):
        return [(x, y) for x in range(self.size) for y in range(self.size)]

    def __str__(self):
        return '\n'.join(' '.join(str(cell) for cell in row) for row in self.board)
```

CREATES A RANDOM BOARD (2D MATRIX)

```python
def create_random_board(size: int,  seed: int, num_toggles: int = None):
    random.seed(time() if seed is None else seed)

    board = [[0 for _ in range(size)] for _ in range(size)]
    puzzle = LightsOutPuzzle(board)

    if num_toggles is None:
        num_toggles = random.randint(1, size * size)

    for _ in range(num_toggles):
        x = random.randint(0, size - 1)
        y = random.randint(0, size - 1)
        puzzle.toggle(x, y)

    return puzzle.board
```

PRINT OUT THE SOLUTION FOR THE PROBLEM AND ITS INFORMATION

```python
def show_solution(
    puzzle: LightsOutPuzzle,
    solution: List[Tuple[int, int]],
    algorithm_name: str,
    nodes_visited: int,
    show_steps: bool = False,
):
    print(f"\nSolving with {algorithm_name}:")
    if solution is None:
        print("No solution found.")
        return
    print(f"Solution: {solution}")
    print(f"Nodes visited: {nodes_visited}")
    if show_steps:
        print("\nSolution steps:")
        for i, move in enumerate(solution):
            print(f"\nStep {i+1}: Toggle position {move}")
            puzzle.toggle(*move)
            print(puzzle)
        print(
            "\nFinal state (solved):"
            if puzzle.is_solved()
            else "\nFinal state (not solved):"
        )
        print(puzzle)
```

CREATING 3 SET OF 3*3 , 4*4 AND 5*5 RANDOM PROBLEMS TO TEST

```
tests = [
    create_random_board(3, 42),
    create_random_board(3, 41),
    create_random_board(3, 42, 5),
    create_random_board(4, 42),
    create_random_board(4, 41),
    create_random_board(4, 42, 5),
    create_random_board(5, 42),
    create_random_board(5, 41),
    create_random_board(5, 42, 5),
]
```

BFS

```python
# TODO: Must return a list of tuples and the number of visited nodes
def bfs_solve(puzzle: LightsOutPuzzle) -> Tuple[List[Tuple[int, int]], int]:
    start_board = puzzle.board.copy()
    # Use a queue for BFS (deque is a double ended quque)
    queue = deque([(start_board, [])])  # (board, path)
    # Use a set to store visited states
    # set is list with only unique value making it useful for
    # checking if it already has been vidited
    visited = set()
    visited.add(str(start_board))

    # Count the number of visited nodes
    nodes_visited = 0

    while queue:

        # current_matrix will contain the initial matrix
        # path will contain the empty list
        current_board, path = queue.popleft()
        nodes_visited += 1

        # If all lamps are off, return the solution
        if np.all(current_board == 0):
            return path, nodes_visited

        # Try toggling each light
        for x in range(puzzle.size):
            for y in range(puzzle.size):
                # New state after pressing the switch (i, j)
                new_board = current_board.copy()
                puzzle.board = new_board
                puzzle.toggle(x, y)
                new_state = puzzle.board

                # Check if the new state has been visited
                if str(new_state) not in visited:
                    visited.add(str(new_state))
                    queue.append((new_state.copy(), path + [(x, y)]))

    return None, nodes_visited  # No solution found
```

we iterate through each element of matrix and after checking the result it (apply toggle) , if it was correct we return path answer otherwise we push it to the end of the stack so we examine sub childs

IDS

```
# Depth-limited DFS helper for IDS
def dfs_limited(puzzle, board, limit, path, visited):
    if np.all(board == 0): #check if solved
        return path, True
    if limit == 0:
        return None, False

    visited.add(str(board)) # Mark current state as visited


    for x in range(puzzle.size):
        for y in range(puzzle.size):
            new_board = board.copy()
            puzzle.board = new_board
            puzzle.toggle(x, y)
            new_state = puzzle.board

            if str(new_state) not in visited: # If not visited
                result, solved = dfs_limited(puzzle, new_state, limit - 1, path + [(x, y)],
visited)
                if solved:
                    return result, solved

    return None, False # no solution
```

IDS is doing iterative DFS with limited depth so first we define limited DFS to implement it in IDS . We create a base case for it when limit is 0 (end) and reclusively call the function call the function to reach its limit and also we save current state in visited to avoid repeating

Similar to BFS it iterates through every possible element

```python
# TODO: Must return a list of tuples and the number of visited nodes
def ids_solve(puzzle: LightsOutPuzzle) -> Tuple[List[Tuple[int, int]], int]:
    start_board = puzzle.board.copy()
    depth = 0
    nodes_visited = 0
    size = puzzle.size
    max_depth = size ** 2  # Set max depth as n^2

    while depth <= max_depth:
        visited = set()
        result, solved = dfs_limited(puzzle, start_board, depth, [], visited)
        nodes_visited += len(visited)
        if solved:
            return result, nodes_visited
        depth += 1

    # If no solution is found within the max depth
    return None, nodes_visited
```

we know max depth is size ^ 2 so we iterate until we find answer or we hit all dead ends

A*

```
# TODO: Must return a list of tuples and the number of visited nodes
def astar_solve(puzzle: LightsOutPuzzle, heuristic: Callable[[LightsOutPuzzle], int]) ->
Tuple[List[Tuple[int, int]], int]:
    start = puzzle.board.tobytes()
    pq = [(heuristic(puzzle), 0, start, [])]  # (priority, cost, state, path)
    visited = set([start])
    nodes_visited = 0

    while pq:
        priority, cost, state, path = heapq.heappop(pq)
        nodes_visited += 1

        current_puzzle = LightsOutPuzzle(np.frombuffer(state,
dtype=np.uint8).reshape(puzzle.size, puzzle.size))

        if current_puzzle.is_solved():
            return path, nodes_visited

        for move in current_puzzle.get_moves():
            new_puzzle = LightsOutPuzzle(current_puzzle.board.copy())
            new_puzzle.toggle(*move)
            new_state = new_puzzle.board.tobytes()

            if new_state not in visited:
                visited.add(new_state)
                new_cost = cost + 1
                new_priority = new_cost + heuristic(new_puzzle)
                heapq.heappush(pq, (new_priority, new_cost, new_state, path + [move]))

    return None, nodes_visited  # No solution found
```

we create a priority queue (pq) we iterate though it until finished or found answer , we fill pq base on

priority = heuristic + cost and explore from low priority to high

For each state we examine next possible moves with get_moves() and afterward we update the cost value +1 . If a new state is unvisited, it is added to the queue with its updated priority

heuristic

```
# TODO: Implement heuristic functions and store them in the list

def heuristic1(puzzle: LightsOutPuzzle):
    return np.sum(puzzle.board)  # Number of lit tiles

heuristics = [heuristic1]
```

we define first one as number of lit lamp so less lamp that's on therefore it's better

```
# Manhattan Distance Heuristic
def heuristic2(puzzle: LightsOutPuzzle):
    # Calculates a consistent heuristic based on the Manhattan distance
    total_distance = 0
    for x in range(puzzle.size):
        for y in range(puzzle.size):
            if puzzle.board[x, y] == 1:
                total_distance += min(
                    abs(x - tx) + abs(y - ty)
                    for tx in range(puzzle.size) for ty in range(puzzle.size)
                )
    return total_distance
```

we define second one as Manhattan Distance which calculate sum of Manhattan Distance for each node

Manhattan Distance for each node is closest distance to next lit node (since distance isn't a great factor here this heuristic might not perform as well as anticipated)

weighted heuristic

```
# TODO: Implement weighted heuristic functions and store them in the list
def weighted_heuristic1(puzzle: LightsOutPuzzle, alpha = 5):
  return heuristic1(puzzle) * alpha

def weighted_heuristic2(puzzle: LightsOutPuzzle, alpha = 2):
  return heuristic2(puzzle) * alpha

weighted_heuristics = [weighted_heuristic1, weighted_heuristic2]

# TODO: assign 2 weights you wanna run
weights = [2, 5]
```

we simply multiply our original heuristic

Result

Prints result of our search and relevant information

Questions

1. نحوه مدل کردن مسئله شامل تعریف initial state، goal state، action و ... را به طور دقیق توضیح دهید.

1. Modeling the Problem

Initial State

The initial state represents the configuration of lamps in the hall, which is a binary matrix of size n×n. Each entry in the matrix can either be: 1 meaning its lit or 0 if its tured off.

Goal State

The given matrix turned into a zero matrix and every element is zero

Actions

Toggle lamp which switches value of corresponding element and its neighbors 1 to 0 and 0 to 1

2. هر یک از الگوریتم‌های پیاده‌سازی شده را توضیح دهید و تفاوت‌ها و مزیت‌های هر یک نسبت به دیگری را قید کرده و عنوان کنید که کدام الگوریتم‌ها جواب بهینه تولید می‌کنند.

Breadth-First Search (BFS)

- **Description: BFS explores all possible configurations level by level, ensuring that the shortest path to the goal is found.**
- **Optimality: Guarantees finding the optimal solution if one exists.**
- **Advantages: Simplicity and guaranteed optimality for unweighted problems.**
- **Disadvantages: High memory consumption due to storing all nodes at the current level and it can be slow.**

Iterative Deepening Search (IDS)

- **Description: IDS combines depth-first search's space efficiency with breadth-first search's completeness. It repeatedly performs depth-limited searches with increasing limits.**
- **Optimality: Guarantees finding the optimal solution if one exists.**
- **Advantages: More space-efficient than BFS and still guarantees finding an optimal solution.**
- **Disadvantages: Repeatedly explores nodes, leading to potentially higher execution time.**

A* Search

- **Description: A* uses a heuristic to prioritize nodes based on the estimated total cost to reach the goal.**
- **Optimality: Guarantees finding the optimal solution when using an admissible heuristic(possible not to find optimal solution).**
- **Advantages: Efficiently directs the search toward the goal using heuristics.**
- **Disadvantages: Performance heavily relies on the choice of the heuristic, answer might not be optimal.**

3. heuristicهای معرفی شده در بخش جستجوی آگاهانه را توضیح داده و هر یک را از نظر admissible بودن و consistent بودن بررسی کنید.

Explanation of process on the heuristic code.

Admissibility: A heuristic is admissible if it never overestimates the cost to reach the goal. In other words, it must always be less than or equal to the true cost of reaching the goal from any state.

Consistency: A heuristic is consistent if, for every node n, the estimated cost of reaching the goal from n is no greater than the cost of reaching any neighboring node n′ plus the estimated cost from n′ to the goal. This can be written as:

$$h(n) \leq c(n,n') + h(n')$$

where c(n,n′) is the actual cost of moving from node n to node n′.

heuristic 1:

This heuristic counts the number of lights that are still on. It never overestimates the number of moves needed to turn off all the lights because turning off one light at a time is the minimum possible cost. Therefore, heuristic 1 is admissible.

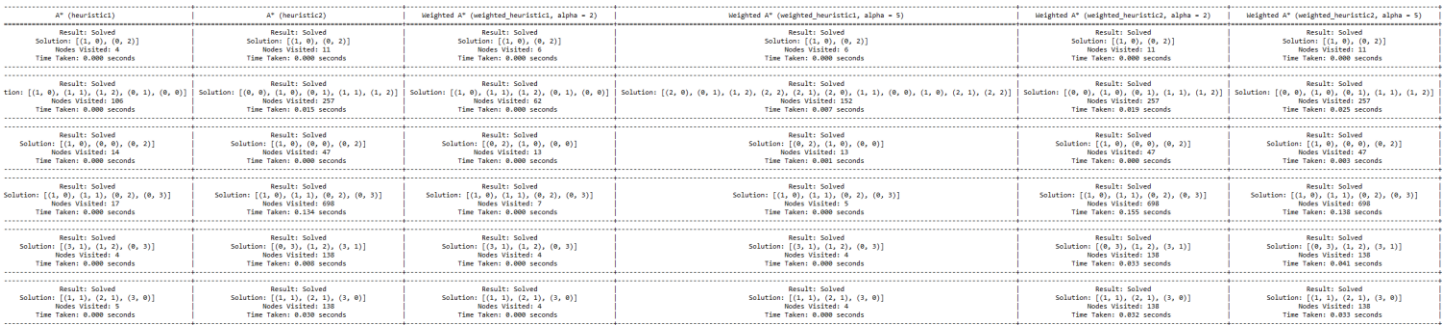
Since pressing a button can only affect up to 5 lights, the heuristic will decrease by 1 or more when we make a valid move. Therefore, the heuristic will always satisfy the condition $h(n) \leq c(n,n') + h(n')$ as the cost to toggle is typically $c(n,n')=1$. Hence, heuristic 1 is also consistent.

heuristic 2:

Since it is a lower bound on the number of moves (it does not account for the fact that a single move toggle multiple lights), it is admissible, as it never overestimates the true cost.

I think it is inconsistent since a twisted combination switches be needed for a possible solution and result in increasing the heuristic

4. سپس، الگوریتم را با استفاده از تمام heuristicهایی که معرفی کردید اجرا کرده و نتایج آن‌ها را با یکدیگر مقایسه کنید.



A* (heuristic1)	A* (heuristic2)	Weighted A* (weighted_heuristic1, alpha = 2)	Weighted A* (weighted_heuristic1, alpha = 5)	Weighted A* (weighted_heuristic2, alpha = 2)	Weighted A* (weighted_heuristic2, alpha = 5)
Result: Solved Solution: [(1, 0), (0, 2)] Nodes Visited: 4 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (0, 2)] Nodes Visited: 11 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (0, 2)] Nodes Visited: 6 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (0, 2)] Nodes Visited: 6 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (0, 2)] Nodes Visited: 11 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (0, 2)] Nodes Visited: 11 Time Taken: 0.000 seconds
Result: Solved tion: [(1, 0), (1, 1), (1, 2), (0, 1), (0, 0)] Nodes Visited: 106 Time Taken: 0.000 seconds	Result: Solved Solution: [(0, 0), (1, 0), (0, 1), (1, 1), (1, 2)] Nodes Visited: 257 Time Taken: 0.015 seconds	Result: Solved Solution: [(1, 0), (1, 1), (1, 2), (0, 1), (0, 0)] Nodes Visited: 62 Time Taken: 0.000 seconds	Result: Solved Solution: [(2, 0), (0, 1), (1, 2), (2, 2), (2, 1), (2, 0), (1, 1), (0, 0), (1, 0), (2, 1), (2, 2)] Nodes Visited: 152 Time Taken: 0.007 seconds	Result: Solved Solution: [(0, 0), (1, 0), (0, 1), (1, 1), (1, 2)] Nodes Visited: 257 Time Taken: 0.019 seconds	Result: Solved Solution: [(0, 0), (1, 0), (0, 1), (1, 1), (1, 2)] Nodes Visited: 257 Time Taken: 0.025 seconds
Result: Solved Solution: [(1, 0), (0, 0), (0, 2)] Nodes Visited: 14 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (0, 0), (0, 2)] Nodes Visited: 47 Time Taken: 0.000 seconds	Result: Solved Solution: [(0, 2), (1, 0), (0, 0)] Nodes Visited: 13 Time Taken: 0.000 seconds	Result: Solved Solution: [(0, 2), (1, 0), (0, 0)] Nodes Visited: 13 Time Taken: 0.001 seconds	Result: Solved Solution: [(1, 0), (0, 0), (0, 2)] Nodes Visited: 47 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (0, 0), (0, 2)] Nodes Visited: 47 Time Taken: 0.003 seconds
Result: Solved Solution: [(1, 0), (1, 1), (0, 2), (0, 3)] Nodes Visited: 17 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (1, 1), (0, 2), (0, 3)] Nodes Visited: 698 Time Taken: 0.134 seconds	Result: Solved Solution: [(1, 0), (1, 1), (0, 2), (0, 3)] Nodes Visited: 7 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (1, 1), (0, 2), (0, 3)] Nodes Visited: 5 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (1, 1), (0, 2), (0, 3)] Nodes Visited: 698 Time Taken: 0.155 seconds	Result: Solved Solution: [(1, 0), (1, 1), (0, 2), (0, 3)] Nodes Visited: 698 Time Taken: 0.138 seconds
Result: Solved Solution: [(3, 1), (1, 2), (0, 3)] Nodes Visited: 4 Time Taken: 0.000 seconds	Result: Solved Solution: [(0, 3), (1, 2), (3, 1)] Nodes Visited: 138 Time Taken: 0.008 seconds	Result: Solved Solution: [(3, 1), (1, 2), (0, 3)] Nodes Visited: 4 Time Taken: 0.000 seconds	Result: Solved Solution: [(3, 1), (1, 2), (0, 3)] Nodes Visited: 4 Time Taken: 0.000 seconds	Result: Solved Solution: [(0, 3), (1, 2), (3, 1)] Nodes Visited: 138 Time Taken: 0.033 seconds	Result: Solved Solution: [(0, 3), (1, 2), (3, 1)] Nodes Visited: 138 Time Taken: 0.041 seconds
Result: Solved Solution: [(1, 1), (2, 1), (3, 0)] Nodes Visited: 5 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 1), (2, 1), (3, 0)] Nodes Visited: 138 Time Taken: 0.030 seconds	Result: Solved Solution: [(1, 1), (2, 1), (3, 0)] Nodes Visited: 4 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 1), (2, 1), (3, 0)] Nodes Visited: 4 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 1), (2, 1), (3, 0)] Nodes Visited: 138 Time Taken: 0.032 seconds	Result: Solved Solution: [(1, 1), (2, 1), (3, 0)] Nodes Visited: 138 Time Taken: 0.033 seconds

A* (heuristic1)	A* (heuristic2)
Result: Solved Solution: [(1, 0), (0, 2)] Nodes Visited: 4 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (0, 2)] Nodes Visited: 11 Time Taken: 0.000 seconds
Result: Solved Solution: [(1, 0), (1, 1), (1, 2), (0, 1), (0, 0)] Nodes Visited: 106 Time Taken: 0.000 seconds	Result: Solved Solution: [(0, 0), (1, 0), (0, 1), (1, 1), (1, 2)] Nodes Visited: 257 Time Taken: 0.015 seconds
Result: Solved Solution: [(1, 0), (0, 0), (0, 2)] Nodes Visited: 14 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (0, 0), (0, 2)] Nodes Visited: 47 Time Taken: 0.000 seconds
Result: Solved Solution: [(1, 0), (1, 1), (0, 2), (0, 3)] Nodes Visited: 17 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 0), (1, 1), (0, 2), (0, 3)] Nodes Visited: 698 Time Taken: 0.134 seconds
Result: Solved Solution: [(3, 1), (1, 2), (0, 3)] Nodes Visited: 4 Time Taken: 0.000 seconds	Result: Solved Solution: [(0, 3), (1, 2), (3, 1)] Nodes Visited: 138 Time Taken: 0.008 seconds
Result: Solved Solution: [(1, 1), (2, 1), (3, 0)] Nodes Visited: 5 Time Taken: 0.000 seconds	Result: Solved Solution: [(1, 1), (2, 1), (3, 0)] Nodes Visited: 138 Time Taken: 0.030 seconds

heuristic 1:

performed well and reduced note as weight increased

heuristic 2:

performed not so well and weight doesn’t change it in a meaningful way

5. به ازای هر الگوریتم، تست‌کیس‌ها را اجرا کرده و میانگین زمان اجرای هر الگوریتم را ثبت کنید. برای هر تست کیس جدول زیر را کامل کنید (برای نمایش جدول زیر می‌توانید از کد موجود در نوت‌بوکی که به شما داده شده‌است استفاده کنید و نیازی به تغییر آن نیست).

The result is saved in Result.txt for better observation (ps: I didn’t have enough time and 5*5 was taking so much time so I didn’t included them I will check them if there is a presentation or something)